fsoft2026 1DD 1: Iteration 3 – Technical Report MedManager v3: A C++ Command-Line Application for Vaccination Center Management and Validation

André Moreira, Gonçalo Oliveira, and Pedro Guilherme

ISEP, Instituto Superior de Engenharia do Porto,
Rua do Dr António Bernardino de Almeida 431, 4249-015 Porto, Portugal
{1240567,1241757,1242036}@isep.ipp.pt
https://www.isep.ipp.pt

Abstract. This technical report presents the third iteration of the MedManager project, a local C++ application developed to support the management of vaccination center operations. The report describes the system's domain model, glossary, class diagram, user interface structure, use case specifications, and sequence diagrams. It also summarizes the current implementation status, including vaccine management, employee and SNS user registration, appointment scheduling, arrival registration, vaccine administration, recovery-room management, data persistence, and report generation. Furthermore, representative code excerpts and automated GoogleTest test cases are presented to demonstrate the implemented functionality and validation mechanisms. The report concludes by identifying remaining improvements, particularly regarding vaccine expiry-date validation, and establishes a solid foundation for the project's final iteration.

Keywords: Business, C++, Vaccination Center, MedManager, Management, Software Development, Command-line Interface, Health

Table of Contents

1 Introduction

The **MedManager** project consists of a local C++ application designed to support the daily operation of a vaccination center. The system centralizes the registration of SNS users, employees, vaccine types, vaccine stock, appointments, arrivals, vaccine administrations, and recovery-room discharge.

This report presents the current state of the project after the second development iteration. It keeps the main architectural artifacts from the previous report, namely the User Interface design, Sequence Diagrams, Domain Model, Class Diagram, and Glossary, while updating the document to reflect the implemented functionality and the tests already created.

The document is written in the third person to describe the system and the development work objectively. Each major section includes an introductory explanation before its subsections, so that the diagrams and tables are not presented without context.

2 Project Status

In this final iteration, the project includes the core Model, Controller, and View structure for the MedManager application. The implemented functionality covers vaccine type creation, vaccine stock registration, employee registration and listing, SNS user registration, appointment scheduling, arrival registration, waiting-room consultation, vaccine administration, recovery-room consultation, discharge from the recovery room, data updates, data persistence, and CSV report exports.

The project also includes an automated test suite using GoogleTest. These tests validate both successful flows and rejection scenarios, such as duplicated vaccine types, duplicated vaccine lots, invalid employee roles, invalid SNS user data, duplicated appointments, users without appointments, wrong vaccine types during administration, and empty export operations.

Although this final release delivers a fully functional architecture, there is a known minor limitation regarding the vaccine expiry date. While the date is properly stored and its format is validated by the existing utility functions, the system does not yet actively block the registration of expired vaccine lots. This aspect remains as a potential future enhancement for the project, though all other critical validation mechanisms and operational workflows are fully complete and tested.

3 Domain Model

The Domain Model of the MedManager system defines and structures the fundamental concepts of the vaccination process. It translates the business requirements into a diagram that clearly shows the system's entities (such as users and nurses) and how they relate to each other, as shown in Figure 1.

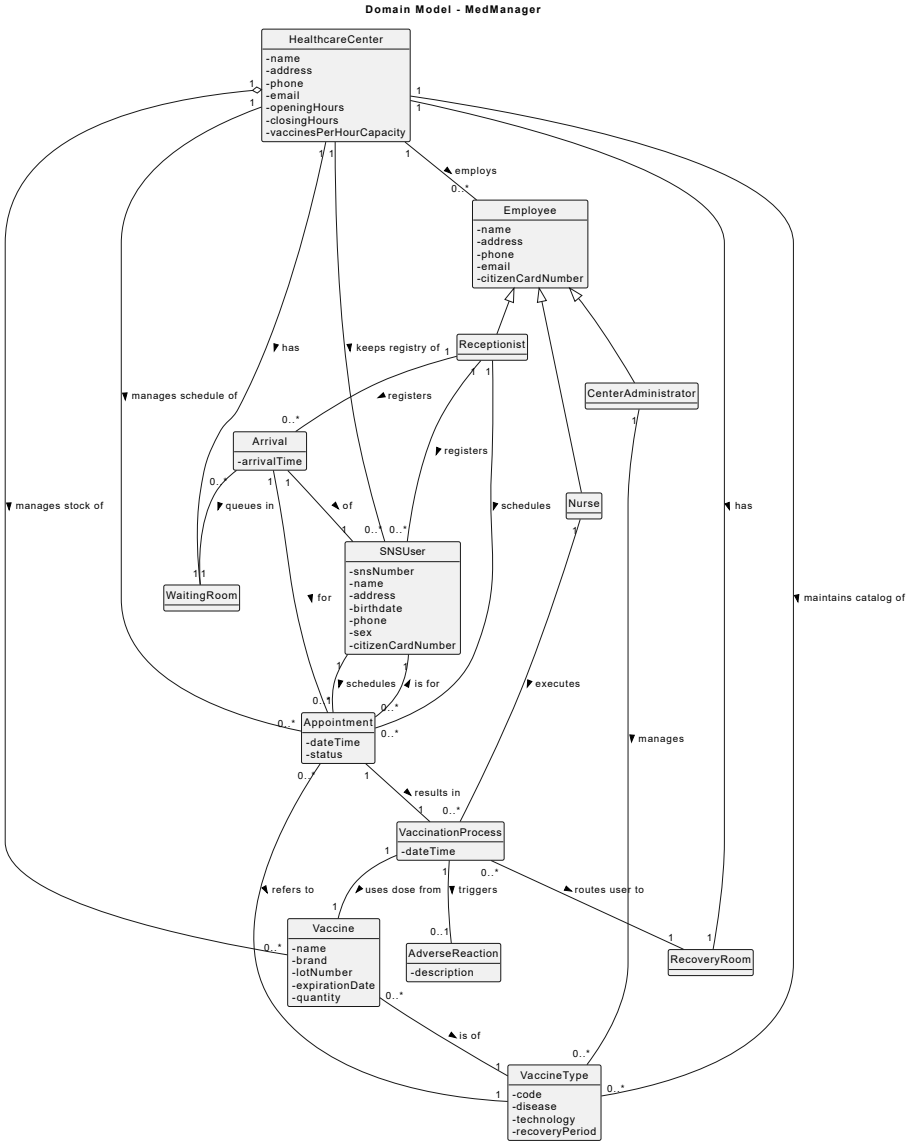

Fig. 1: MedManager's Domain Model

4 Glossary

Table 1: MedManager Glossary

Term/Expression (EN)	Termo (PT)	Definition/Description
Adverse Reaction	Reação Adversa	Clinical event or side effect recorded after vaccination, typically monitored in the recovery room during the designated period.
Appointment	Agendamento	A scheduled slot (date and time) for an SNS User to receive a specific vaccine type. It serves as the prerequisite for a vaccination process.
Arrival	Chegada	The formal record of the user's physical presence at the center. It links an Appointment to the Waiting Room, triggering the service flow.
Center Administrator	Administrador do Centro	Employee responsible for managing center resources, including the vaccine catalog (types), physical stock, and employee records.
Employee	Funcionário	Any worker at the center (Administrator, Nurse, or Receptionist), identified by name, address, phone, email, and citizen card number.
Healthcare Center	Centro de Saúde	The facility where vaccinations occur, acting as the central management entity for all users, employees, inventory, and schedules.
Nurse	Enfermeiro	Healthcare professional responsible for the clinical act of vaccination and for consulting the Waiting Room to call the next user.
Receptionist	Rececionista	Employee responsible for registering the arrival of users, confirming appointments, and managing initial user data.
Recovery Room	Sala de Recuperação	A monitored area where users must remain after vaccination for a period defined by the specific Vaccine Type to check for reactions.
SNS User	Utente do SNS	A citizen eligible for vaccination, identified by a unique SNS number, citizen card number, and birthdate.
Vaccination Process	Processo de Vacinação	The clinical act of administering a vaccine dose, linking a nurse, a specific vaccine lot, and an appointment.
Vaccine	Vacina	A physical unit in the center's inventory, characterized by a commercial name, brand, lot number, expiration date, and available quantity.
Vaccine Type	Tipo de Vacina	A category (e.g., COVID-19) that defines the technology used, the disease it prevents, and the required recovery period for users.
Waiting Room	Sala de Espera	A logical queue that manages arrivals in a First-Come, First-Served (FCFS) order until a nurse is available to perform the vaccination.

5 Design

5.1 Model - Class Diagram

Figure 2 maps out the core structural entities of the healthcare center system, detailing their attributes, operations, and relationships to define how clinical data and domain logic are stored and managed.

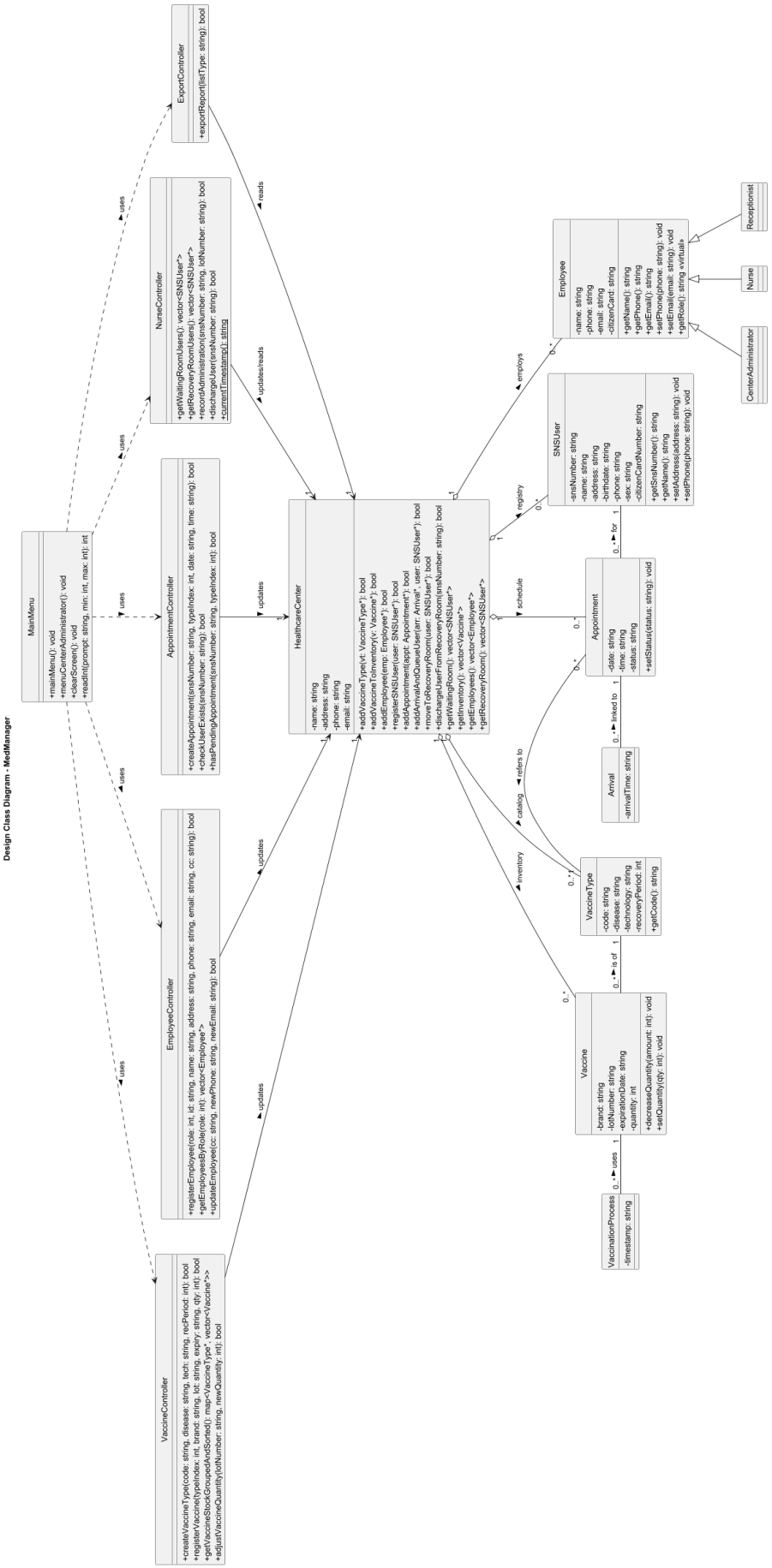

Fig. 2: Model - Class Diagram

5.2 User Interface

Figure 3 represents the general structure of the menus and their interactions within the system.

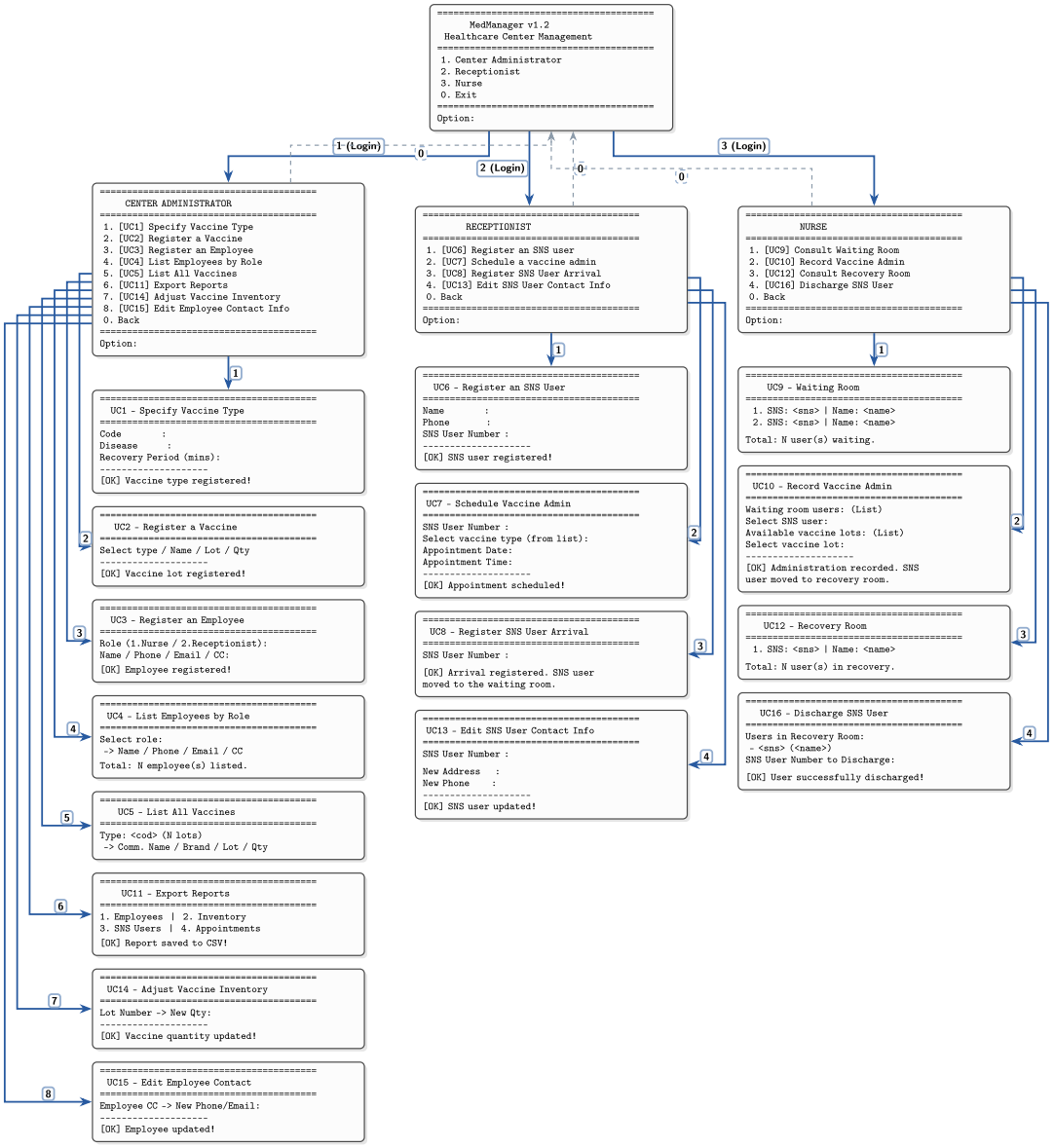

Fig. 3: User Interface Design

5.3 Use Case Diagrams

Figures 4, 5, and 6 provide a high-level visual representation of the system's functionalities, mapping out the interactions separated by the different actors (Center Administrator, Nurse, and Receptionist) within the MedManager application.

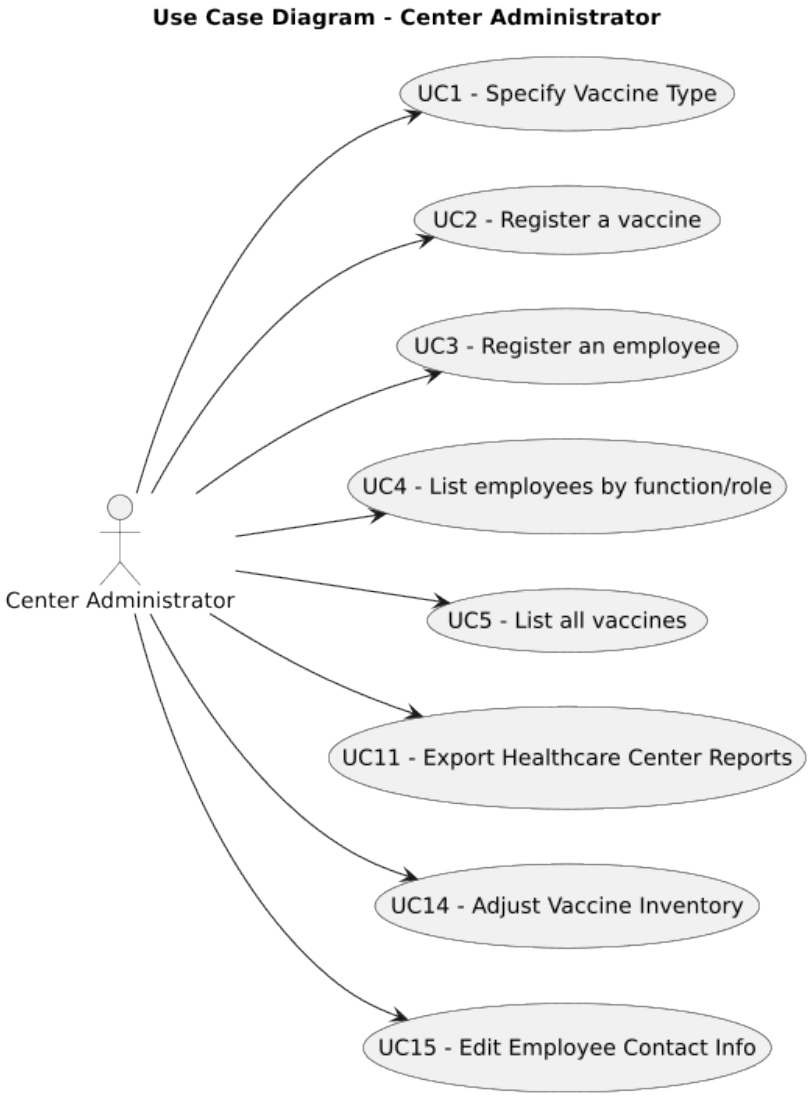

Fig. 4: Use Case Diagram: Center Administrator

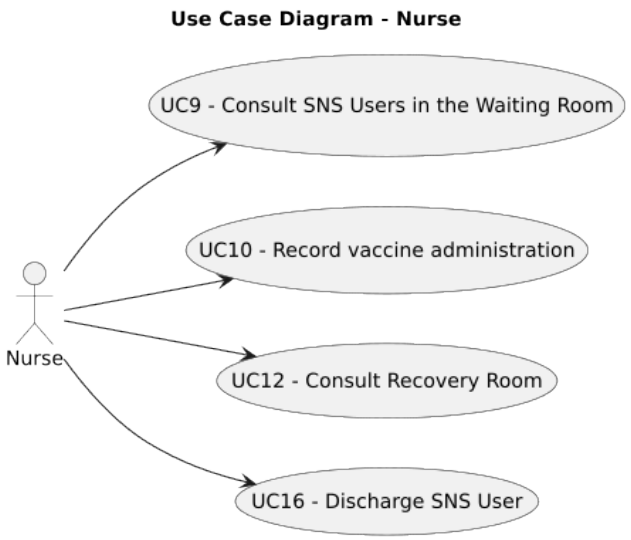

Fig. 5: Use Case Diagram: Nurse

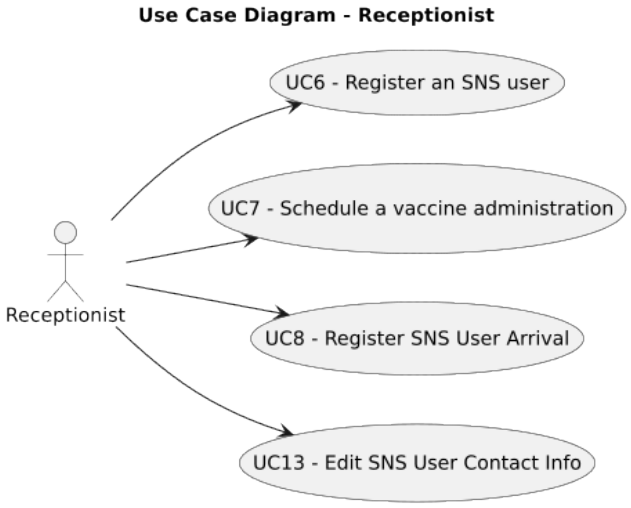

Fig. 6: Use Case Diagram: Receptionist

5.4 Sequence Diagrams

Use Case 1: Create Vaccine Type

The specifications for this use case are detailed in Table 2, and its execution flow is visually represented in the sequence diagram in Figure 7.

Table 2: Use Case 1: Create Vaccine Type

Actor	Center Administrator
Use case name	Create Vaccine Type
Description	The Center Administrator specifies a new vaccine type to be added to the healthcare center's catalog.
Precondition	The user must be logged in as Center Administrator.
Post-condition	A new Vaccine Type is created and added to the vaccine catalog.
Main flow	1. User starts specifying a new vaccine type. 2. System requests data (code, disease, recoveryPeriod) and presents a list of technologies. 3. User inputs data (code, disease, recoveryPeriod) and selects a technology. 4. System validates the code uniqueness, registers the new vaccine type in the catalog, and confirms successful creation.
Alternative path	If the code validation fails (i.e., the vaccine code already exists), the system informs the user and resumes at step 2 (or aborts the operation).

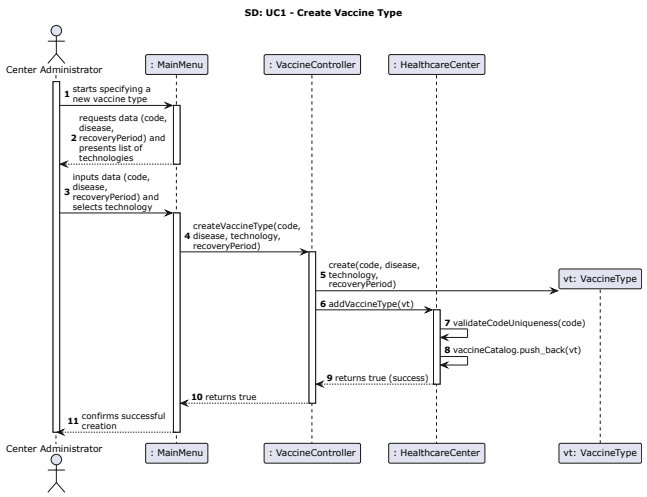

Fig. 7: UC1: Create Vaccine Type

Use Case 2: Register Vaccine

The specifications for this use case are detailed in Table 3, and its execution flow is represented in Figure 8.

Table 3: Use Case 2: Register Vaccine

Actor	Center Administrator
Use case name	Register Vaccine
Description	The Center Administrator registers a batch of specific vaccines into the center's inventory, linking it to an existing vaccine type.
Precondition	The user must be logged in as Center Administrator and the corresponding Vaccine Type must already be registered.
Post-condition	A new batch of vaccines is registered and added to the center's inventory.
Main flow	1. User starts the vaccine registration. 2. System fetches the vaccine catalog and shows a list of available Vaccine Types. 3. User selects the desired vaccine type and inputs the batch data (brand, lot, expiry, quantity). 4. System creates the vaccine batch, adds it to the inventory, and confirms successful registration.
Alternative path	If any inputted data format is invalid or required fields are missing, the system informs the user and resumes at step 2.

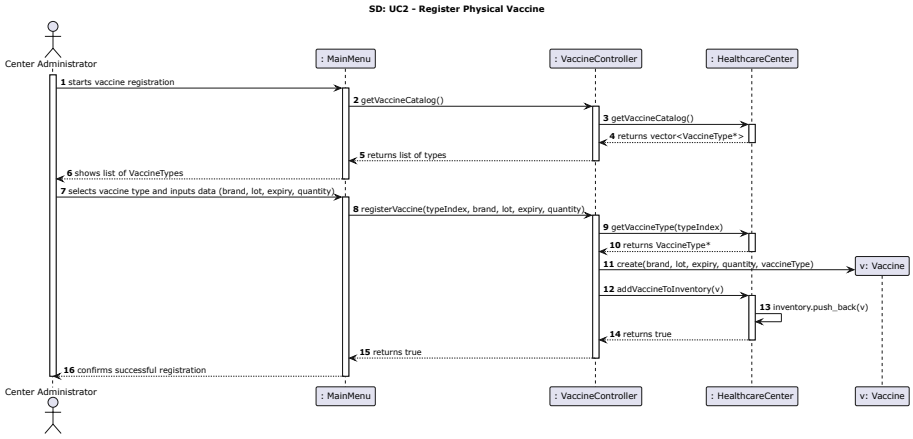

Fig. 8: UC2: Register Physical Vaccine

Use Case 3: Register New Employee

The specifications for this use case are detailed in Table 4, and its execution flow is represented in Figure 9.

Table 4: Use Case 3: Register New Employee

Actor	Center Administrator
Use case name	Register New Employee
Description	The Center Administrator registers a new center staff member, specifying their institutional role (Nurse or Receptionist).
Precondition	The user must be logged in as Center Administrator.
Post-condition	A new Employee profile (Nurse or Receptionist) is registered and added to the center's staff records.
Main flow	1. User starts the registration of a new employee. 2. System shows the available roles (Nurse, Receptionist) and asks to select one. 3. User selects the intended role. 4. System requests personal data (id, name, address, phone, email, cc). 5. User inputs the requested data. 6. System checks for duplicates and creates the employee entity according to the selected role. 7. System adds the new employee to the records and confirms successful registration.
Alternative path	If the system detects duplicated data (e.g., phone, email, or cc already exists), the registration is aborted and an error is displayed.

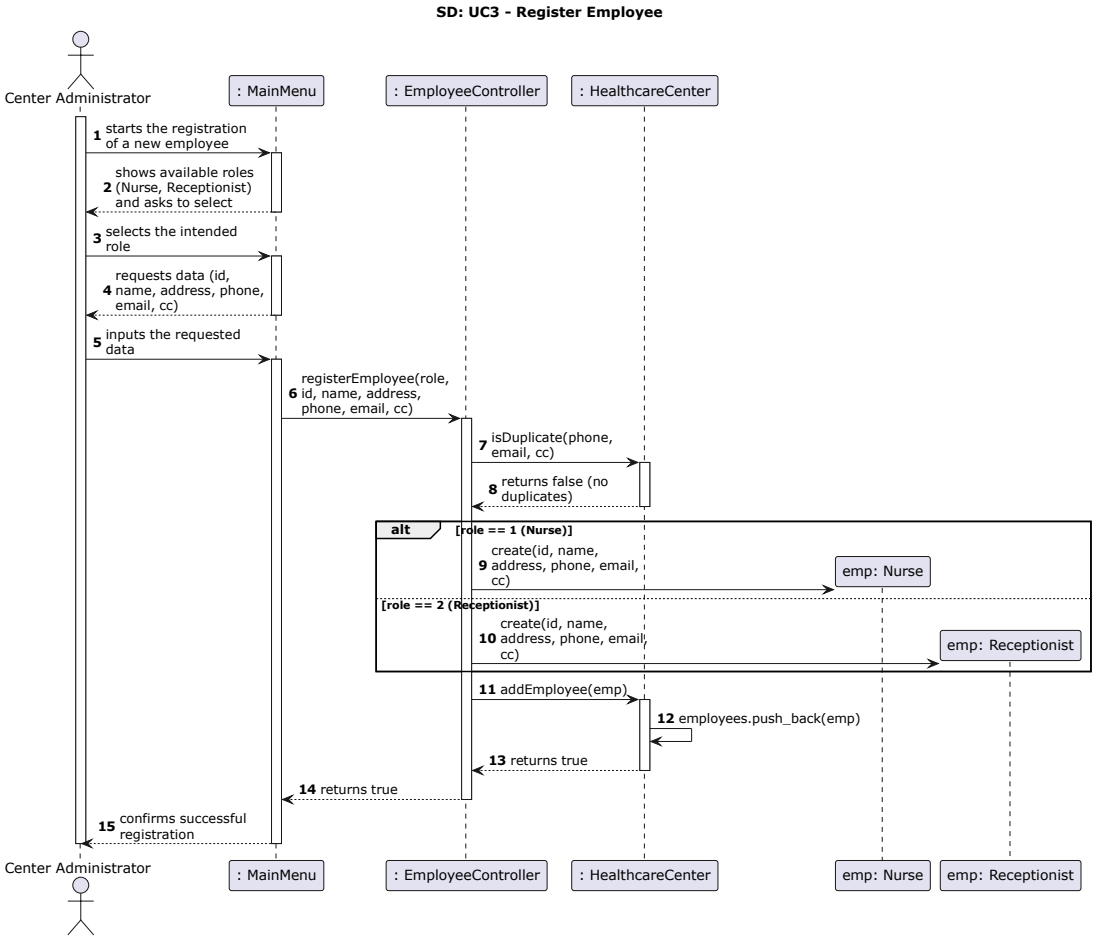

Fig. 9: UC3: Register Employee

Use Case 4: List Employees by Role

The specifications for this use case are detailed in Table 5, and its execution flow is represented in Figure 10.

Table 5: Use Case 4: List Employees by Role

Actor	Center Administrator
Use case name	List Employees by Role
Description	The Center Administrator requests and views a list of registered employees filtered by a specific role (Nurse, Receptionist, or Admin).
Precondition	The user must be logged in as Center Administrator.
Post-condition	The system presents the filtered list of employees or informs that no matches were found.
Main flow	1. User requests a list of employees. 2. System prompts the user to select a role (Nurse, Receptionist, Admin). 3. User selects the desired role. 4. System fetches all employees, filters them by the selected role through an internal loop, and returns a filtered list. 5. System displays the filtered list showing each employee's details (Name, Phone, E-mail, CC).
Alternative path	If the filtered list is empty, the system informs the user that no employees are registered with that specific role.

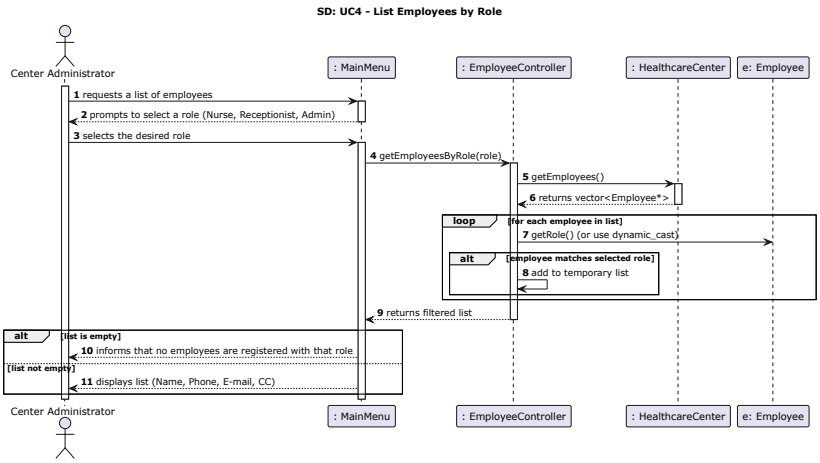

Fig. 10: UC4: List Employees by Role

Use Case 5: List Vaccine Stock Grouped and Sorted

The specifications for this use case are detailed in Table 6, and its execution flow is represented in Figure 11.

Table 6: Use Case 5: List Vaccine Stock Grouped and Sorted

Actor	Center Administrator
Use case name	List Vaccine Stock Grouped and Sorted
Description	The Center Administrator requests and views a list of all vaccines in stock, grouped by vaccine type and sorted alphabetically by brand.
Precondition	The user must be logged in as Center Administrator.
Post-condition	The system presents the structured list of vaccines or informs that the inventory is empty.
Main flow	1. User requests a list of all vaccines. 2. System fetches the inventory and checks if it contains items. 3. System groups the vaccines by VaccineType and sorts each group alphabetically by Brand. 4. System displays the vaccines grouped by type, including the Commercial Name, Brand, Lot Number, Expiration Date, and Quantity for each.
Alternative path	If the inventory is empty, the system returns an empty structure and displays a "No inventory found" message.

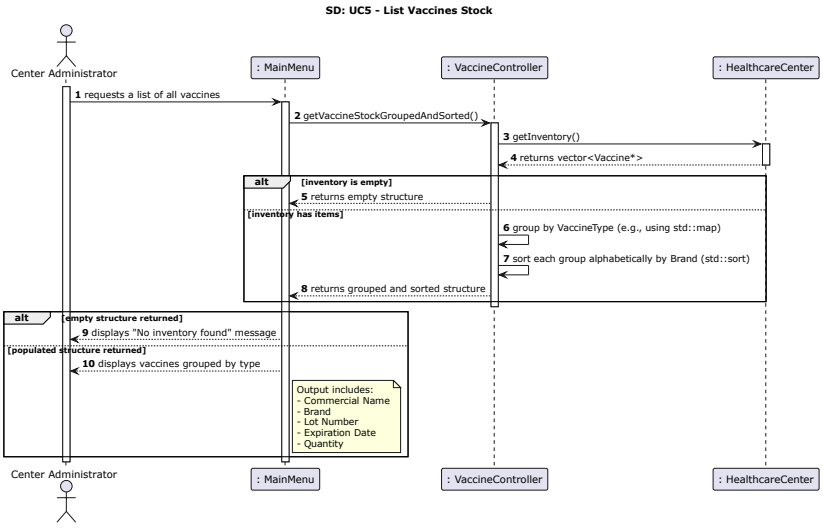

Fig. 11: UC5: List Vaccines Stock

Use Case 6: Register New SNS User

The specifications for this use case are detailed in Table 7, and its execution flow is represented in Figure 12.

Table 7: Use Case 6: Register New SNS User

Actor	Receptionist
Use case name	Register New SNS User
Description	The Receptionist registers a new National Health Service (SNS) user profile in the system's database.
Precondition	The user must be logged in as Receptionist.
Post-condition	A new SNS User profile is created and successfully stored in the health center registry.
Main flow	1. User starts the registration of a new SNS user. 2. System requests personal data (Name, Address, Phone, SNS Number, CC, Birth Date, Sex [Optional]). 3. User inputs the requested data. 4. System checks for user duplication by SNS number and phone, and validates the Portuguese contact/identity formats. 5. System creates the SNS user entity, appends it to the registry records, and confirms successful registration.
Alternative path	If the system detects an invalid format or an existing duplicate, it returns a validation error and informs the user.

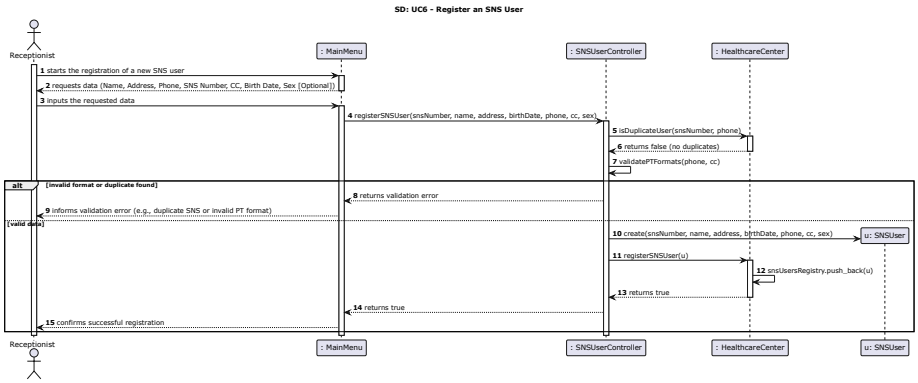

Fig. 12: UC6: Register an SNS User

Use Case 7: Schedule Vaccine Administration

The specifications for this use case are detailed in Table 8, and its execution flow is represented in Figure 13.

Table 8: Use Case 7: Schedule Vaccine Administration

Actor	SNS User
Use case name	Schedule Vaccine Administration
Description	The SNS User schedules a vaccine administration by verifying their registration, choosing a vaccine type, and selecting an available date and time.
Precondition	The user must be registered in the system as a valid SNS User.
Post-condition	A new appointment is successfully scheduled and stored in the healthcare center's master schedule.
Main flow	1. User starts scheduling a vaccine administration. 2. System requests the SNS User Number. 3. User inputs the SNS User Number. 4. System verifies that the user exists, retrieves the vaccine catalog, and presents a list of available Vaccine Types. 5. User selects a vaccine type and inputs the desired Date and Time. 6. System validates the selected date/time, displays an appointment summary, and requests confirmation. 7. User confirms the appointment. 8. System links the appointment details, creates the appointment record, appends it to the master schedule, and confirms successful scheduling.
Alternative path	- If the user is not registered, the system informs a user not found error and aborts. - If the date/time is in the past, the system informs an invalid date/time error.

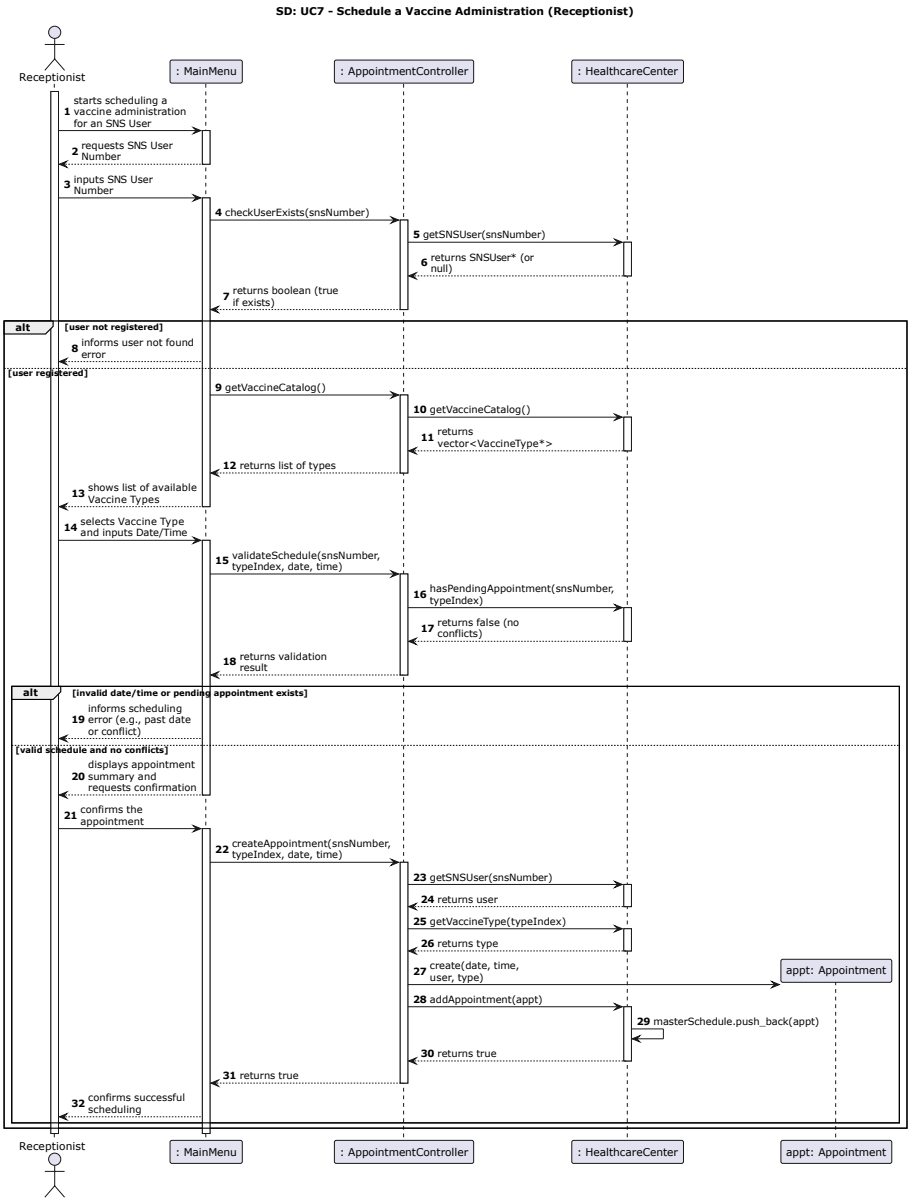

Fig. 13: UC7: Schedule a Vaccine Administration (SNS User)

Use Case 8: Schedule Vaccine Administration for an SNS User

The specifications for this use case are detailed in Table 9, and its execution flow is represented in Figure 14.

Table 9: Use Case 8: Schedule Vaccine Administration for an SNS User

Actor	Receptionist
Use case name	Schedule Vaccine Administration for an SNS User
Description	The Receptionist schedules a vaccine administration on behalf of an SNS User, verifying their identity and ensuring no pending appointments conflict with the selection.
Precondition	The user must be logged in as Receptionist and the target SNS User must exist in the system database.
Post-condition	A new appointment is created and registered in the healthcare center's master schedule.
Main flow	1. User starts scheduling a vaccine administration for an SNS User. 2. System requests the SNS User Number. 3. User inputs the requested SNS User Number. 4. System validates the user's existence, retrieves the available Vaccine Types, and displays them. 5. User selects the intended Vaccine Type and inputs the Date and Time. 6. System checks for scheduling conflicts or pending appointments, displays an appointment summary, and asks for confirmation. 7. User confirms the appointment parameters. 8. System instantiates the new appointment record, stores it in the master schedule, and confirms successful scheduling.
Alternative path	- If the target SNS User is not registered, the system returns a user not found error. - If the date/time is invalid or a pending appointment already exists for that user and vaccine type, the system displays a scheduling error.

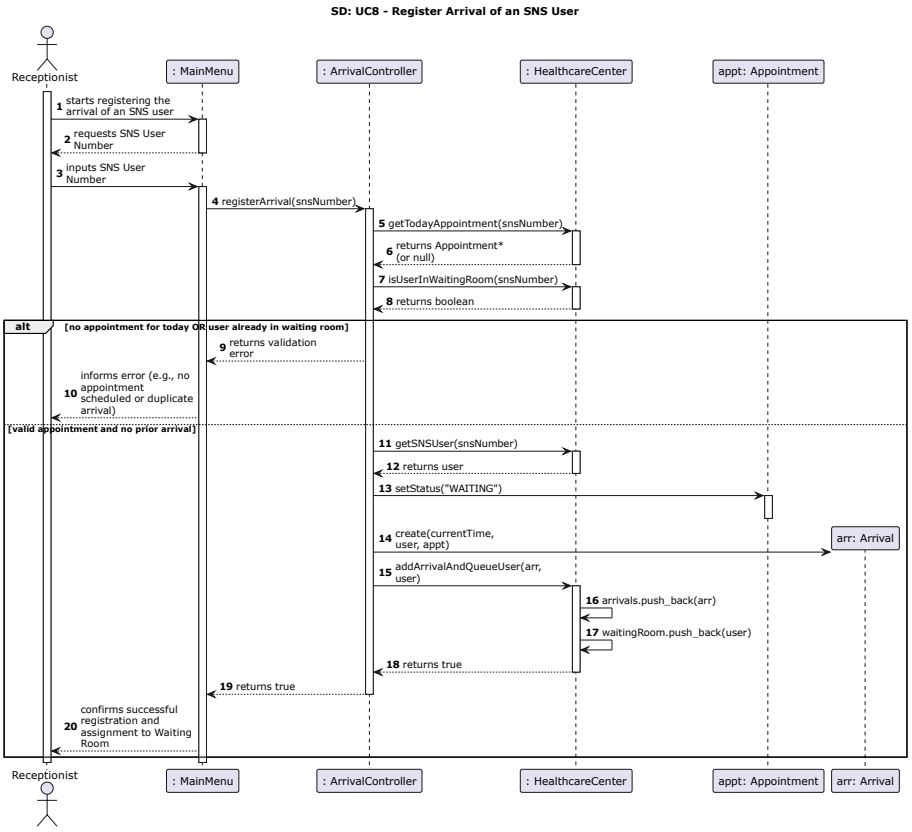

Fig. 14: UC8: Schedule a Vaccine Administration (Receptionist)

Use Case 9: Register Arrival of an SNS User

The specifications for this use case are detailed in Table 10, and its execution flow is represented in Figure 15.

Table 10: Use Case 9: Register Arrival of an SNS User

Actor	Receptionist
Use case name	Register Arrival of an SNS User
Description	The Receptionist registers the arrival of an SNS User at the healthcare center, confirming they have an appointment for the current day and are not already in the waiting room.
Precondition	The user must be logged in as Receptionist.
Post-condition	The user's status is updated to waiting, an arrival record is created, and the user is added to the waiting room queue.
Main flow	1. User starts registering the arrival of an SNS user. 2. System requests the SNS User Number. 3. User inputs the requested SNS User Number. 4. System checks if there is an appointment scheduled for today and verifies that the user is not already in the waiting room. 5. System updates the appointment status to "WAITING". 6. System creates an arrival record with the current time. 7. System adds the arrival to the center's registry, moves the user into the waiting room queue, and confirms successful registration and assignment.
Alternative path	If the user has no appointment scheduled for today or is already registered in the waiting room, the system returns a validation error and informs the user.

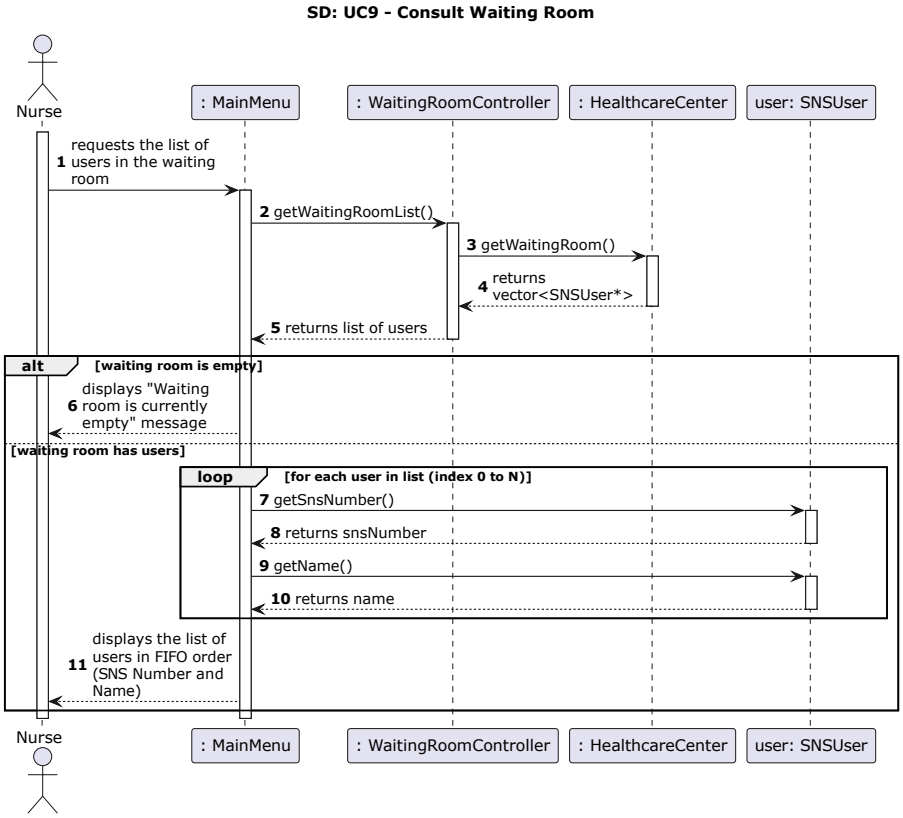

Fig. 15: UC9: Register Arrival of an SNS User

Use Case 10: List Users in the Waiting Room

The specifications for this use case are detailed in Table 11, and its execution flow is represented in Figure 16.

Table 11: Use Case 10: List Users in the Waiting Room

Actor	Nurse
Use case name	List Users in the Waiting Room
Description	The Nurse requests and views the list of SNS Users currently inside the healthcare center's waiting room, displayed in first-in, first-out (FIFO) order.
Precondition	The user must be logged in as Nurse.
Post-condition	The system displays the active queue of waiting users or informs that the waiting room is empty.
Main flow	1. User requests the list of users in the waiting room. 2. System fetches the current waiting room list from the healthcare center data records. 3. System processes each user in the list through a loop to retrieve their identification attributes (SNS Number and Name). 4. System displays the complete list of users in FIFO order, showing each individual's SNS Number and Name.
Alternative path	If the waiting room is empty, the system displays a "Waiting room is currently empty" message.

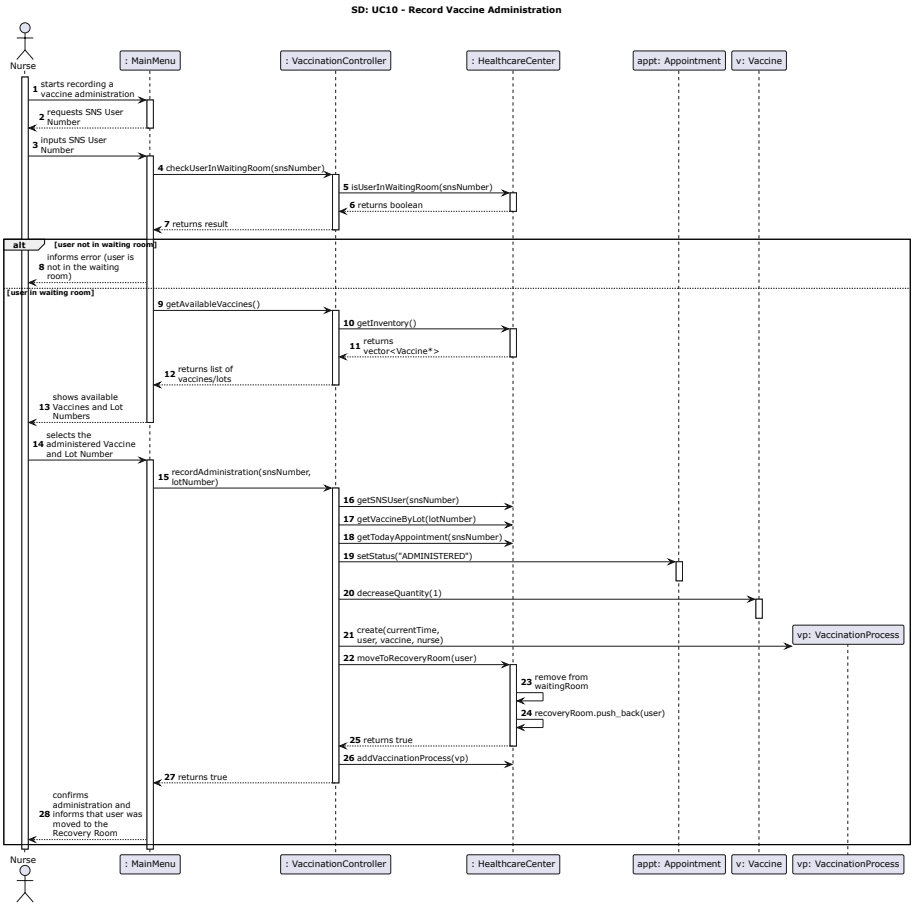

Fig. 16: UC10: Consult Waiting Room

Use Case 11: Record Vaccine Administration

The specifications for this use case are detailed in Table 12, and its execution flow is represented in Figure 17.

Table 12: Use Case 11: Record Vaccine Administration

Actor	Nurse
Use case name	Record Vaccine Administration
Description	The Nurse verifies the presence of an SNS User in the waiting room, selects an available vaccine lot, and records the administration, automatically transitioning the user to the recovery room.
Precondition	The user must be logged in as Nurse.
Post-condition	The vaccine stock quantity is decremented, the appointment status is updated to administered, a vaccination process record is stored, and the user is transferred to the recovery room.
Main flow	1. User starts recording a vaccine administration. 2. System requests the SNS User Number. 3. User inputs the SNS User Number. 4. System checks if the user is currently in the waiting room, fetches the available vaccine stock inventory, and displays a list of Vaccines and Lot Numbers. 5. User selects the administered Vaccine and Lot Number. 6. System updates the user's appointment status to administered and decreases the vaccine stock quantity by 1. 7. System creates a new VaccinationProcess record with the current time and links it to the center's records. 8. System removes the user from the waiting room, adds them to the recovery room queue, and confirms successful administration and transfer.
Alternative path	If the system detects that the user is not in the waiting room, it displays an error message and terminates the process.

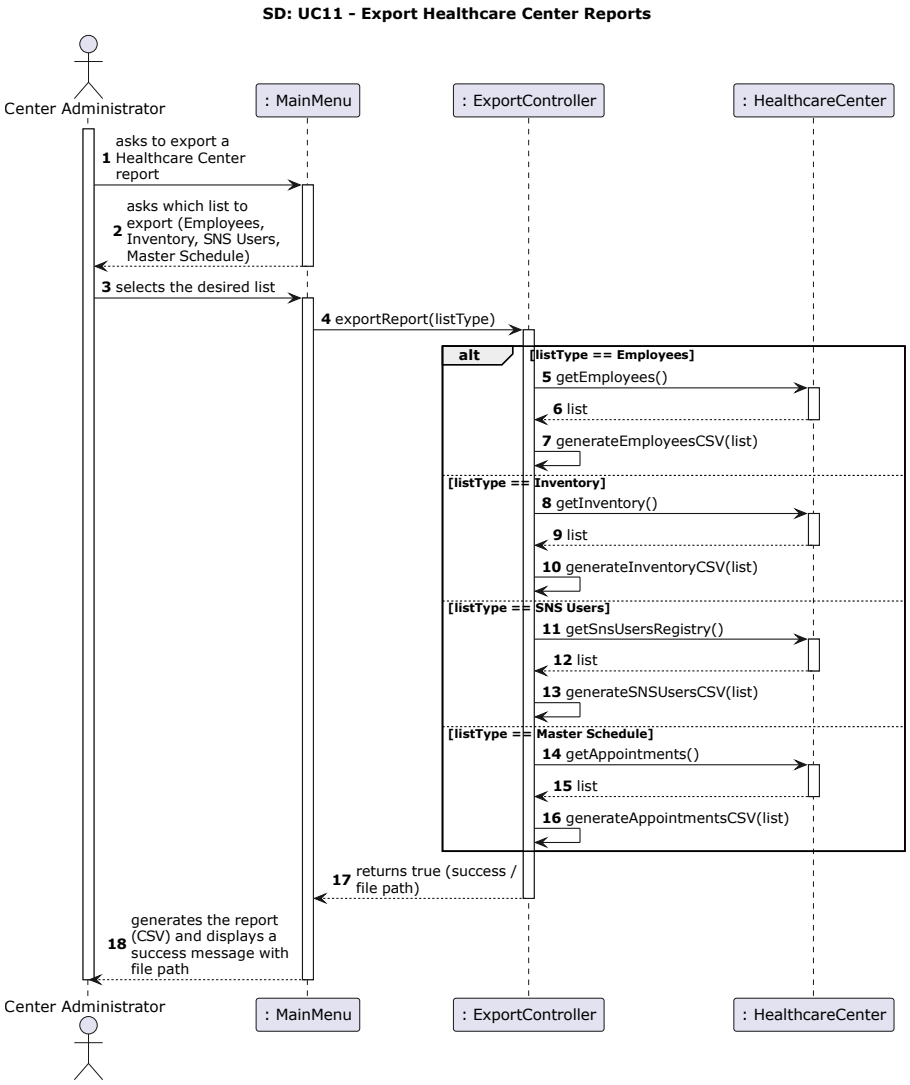

Fig. 17: UC11: Record Vaccine Administration

Use Case 12: Consult Recovery Room

The specifications for this use case are detailed in Table 13, and its execution flow is represented in Figure 18.

Table 13: Use Case 12: Consult Recovery Room

Actor	Nurse
Use case name	Consult Recovery Room
Description	The Nurse requests and views the list of SNS Users currently in the recovery room, checking their remaining recovery time.
Precondition	The user must be logged in as Nurse.
Post-condition	The system displays the active list of users in the recovery room.
Main flow	1. User requests the list of users in the recovery room. 2. System fetches the current recovery room list from the records. 3. System processes each user in the list through a loop to retrieve their identification attributes and administration time. 4. System displays the complete list of users, showing each individual's SNS Number, Name, and elapsed recovery time.
Alternative path	If the recovery room is empty, the system displays a "Recovery room is currently empty" message.

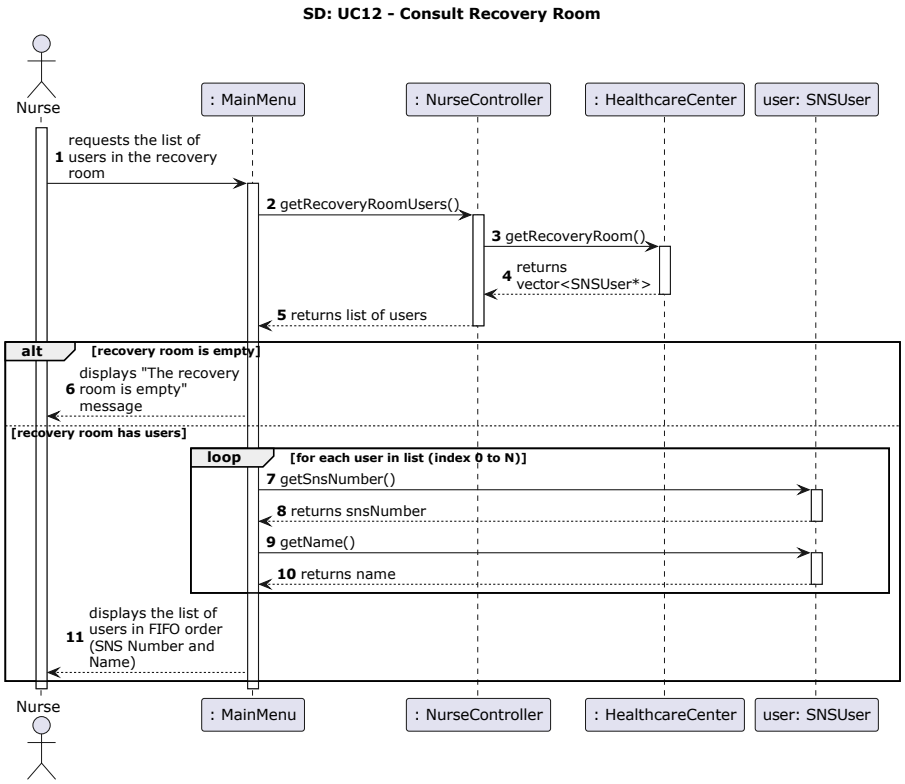

Fig. 18: UC12: Consult Recovery Room

Use Case 13: Edit SNS User Contact Info

The specifications for this use case are detailed in Table 14, and its execution flow is represented in Figure 19.

Table 14: Use Case 13: Edit SNS User Contact Info

Actor	Receptionist
Use case name	Edit SNS User Contact Info
Description	The Receptionist updates the contact information (such as phone number, address, or email) of an existing SNS User.
Precondition	The user must be logged in as Receptionist and the SNS User must be registered.
Post-condition	The SNS User's contact details are successfully updated in the registry.
Main flow	1. User starts the process to edit an SNS User's contact info. 2. System requests the SNS User Number. 3. User inputs the SNS User Number. 4. System validates the user's existence and displays the current contact details. 5. User inputs the new contact data (e.g., Phone, Email, Address). 6. System validates the new format (no duplicates for phone/email), updates the user's record, and confirms the successful edit.
Alternative path	- If the SNS User is not found, the system displays an error and aborts. - If the new contact data is invalid or already registered to another user, the system warns the user and asks to re-enter.

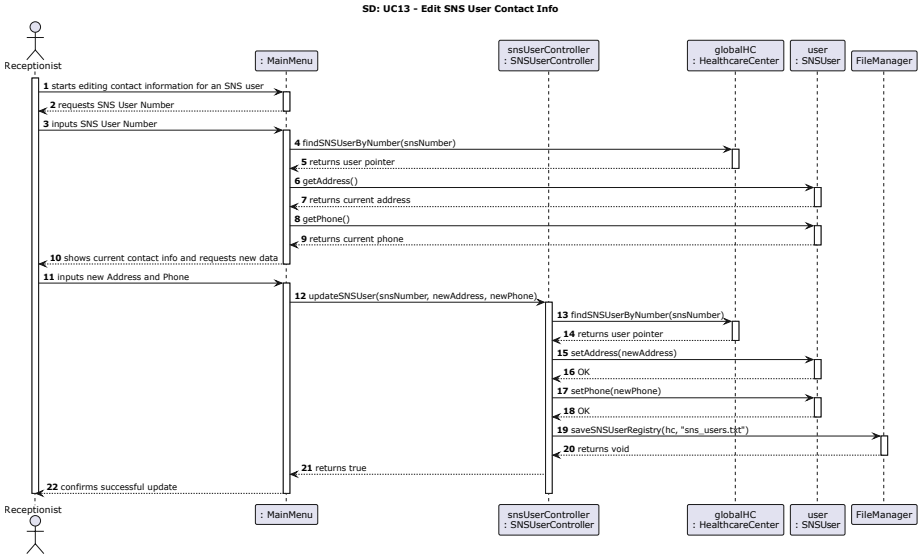

Fig. 19: UC13: Edit SNS User Contact Info

Use Case 14: Adjust Vaccine Inventory

The specifications for this use case are detailed in Table 15, and its execution flow is represented in Figure 20.

Table 15: Use Case 14: Adjust Vaccine Inventory

Actor	Center Administrator
Use case name	Adjust Vaccine Inventory
Description	The Center Administrator manually adjusts the stock quantity of a specific vaccine batch (e.g., due to expiration, damage, or recount).
Precondition	The user must be logged in as Center Administrator.
Post-condition	The system updates the available stock quantity for the specified vaccine batch.
Main flow	1. User starts the inventory adjustment process. 2. System fetches and displays the list of available vaccines grouped by type. 3. User selects the specific Vaccine and Lot Number, and inputs the new quantity along with a justification. 4. System validates the input, overrides the current stock quantity, logs the adjustment reason, and confirms success.
Alternative path	If the entered quantity is invalid (e.g., negative value), the system returns an error and asks the user to input a valid number.

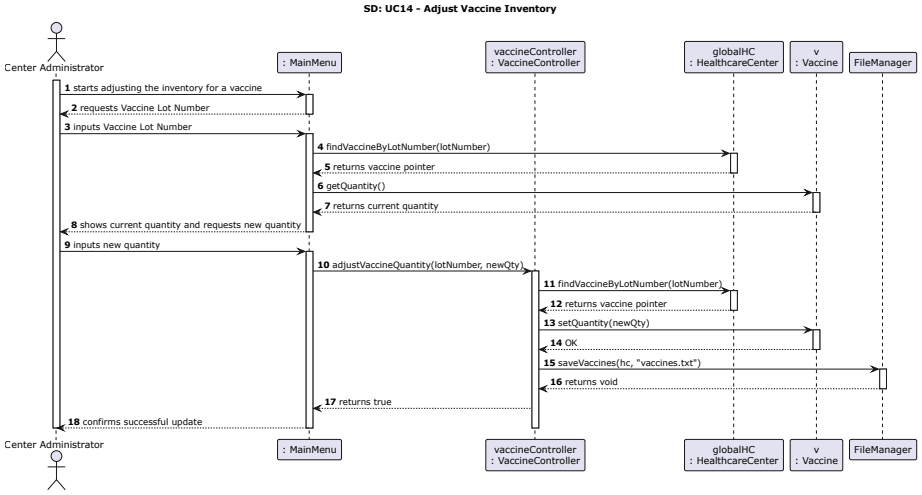

Fig. 20: UC14: Adjust Vaccine Inventory

Use Case 15: Edit Employee Contact Info

The specifications for this use case are detailed in Table 16, and its execution flow is represented in Figure 21.

Table 16: Use Case 15: Edit Employee Contact Info

Actor	Center Administrator
Use case name	Edit Employee Contact Info
Description	The Center Administrator updates the contact information of a registered staff member (Nurse or Receptionist).
Precondition	The user must be logged in as Center Administrator and the target employee must exist.
Post-condition	The employee's contact details are updated in the staff records.
Main flow	1. User starts the process to edit an employee's details. 2. System requests the Employee ID or CC. 3. User inputs the requested identifier. 4. System validates the employee's existence and displays the current personal data. 5. User inputs the new contact data (e.g., Phone, Email, Address). 6. System verifies there are no duplicates for the new data, updates the employee record, and confirms the successful edit.
Alternative path	If the system detects duplicated data (e.g., the new phone number belongs to another employee), the operation is aborted and an error is shown.

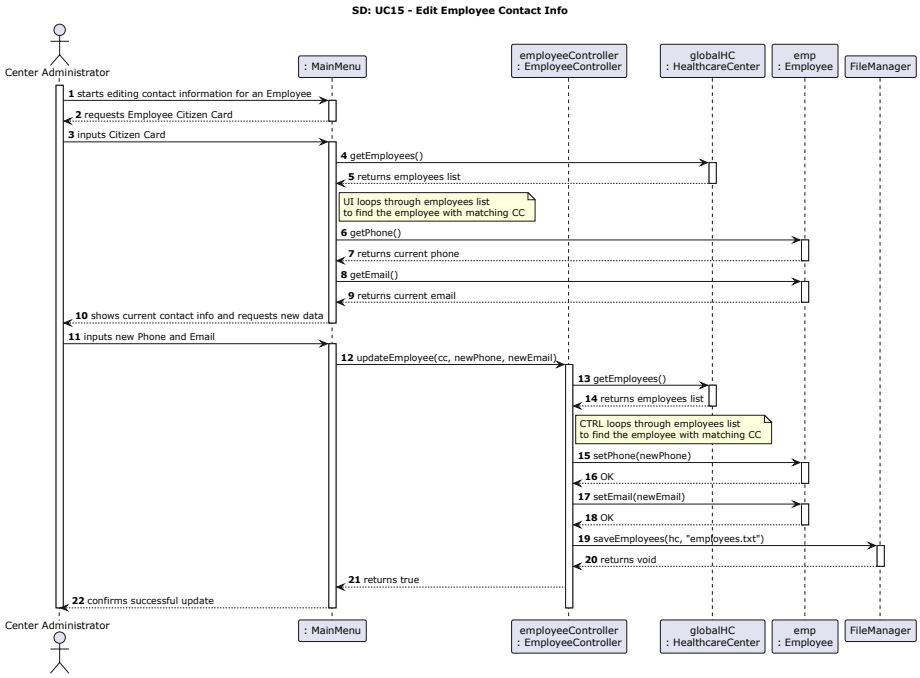

Fig. 21: UC15: Edit Employee Contact Info

Use Case 16: Discharge SNS User

The specifications for this use case are detailed in Table 17, and its execution flow is represented in Figure 22.

Table 17: Use Case 16: Discharge SNS User

Actor	Nurse
Use case name	Discharge SNS User
Description	The Nurse discharges an SNS User from the center after verifying that the mandatory recovery period post-vaccination has been completed safely.
Precondition	The user must be logged in as Nurse and the SNS User must be in the recovery room.
Post-condition	The SNS User is removed from the recovery room queue and their appointment process is closed.
Main flow	1. User starts the discharge process for an SNS User. 2. System requests the SNS User Number. 3. User inputs the SNS User Number. 4. System checks if the user is in the recovery room and verifies if the mandatory recovery time (e.g., 30 minutes) has elapsed. 5. System removes the user from the recovery room queue, updates their final status, and confirms successful discharge.
Alternative path	- If the user is not found in the recovery room, the system returns an error. - If the mandatory recovery time has not yet elapsed, the system warns the Nurse and asks for override confirmation or aborts.

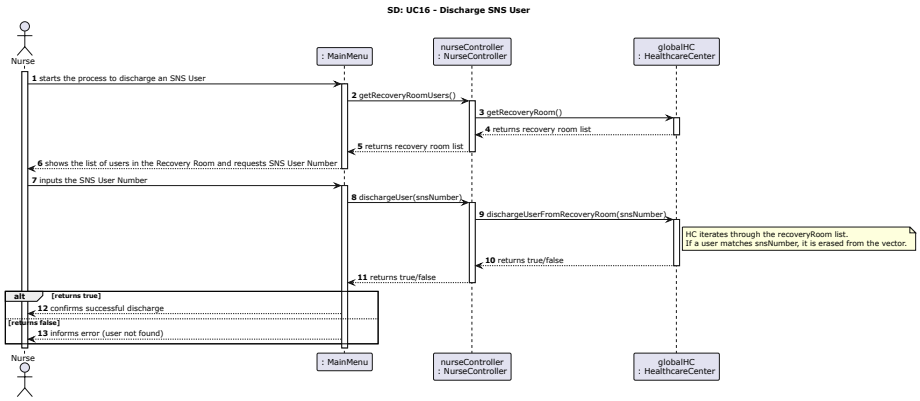

Fig. 22: UC16: Discharge SNS User

6 Implementation and Testing

This section presents representative excerpts from the implemented C++ code and from the GoogleTest test suite. The selected excerpts demonstrate how the system validates data, delegates operations through controllers, updates the domain model, persists relevant changes, and verifies behavior through automated tests.

6.1 Vaccine Type and Vaccine Registration

The following excerpt shows how the controller creates a Vaccine Type and how a physical vaccine lot is linked to a selected type. It also shows the duplicate-handling strategy: if the model rejects an object, the controller deletes the allocated object to avoid keeping invalid data in memory.

Listing 1.1: Vaccine type and vaccine registration implementation

```
bool VaccineController::createVaccineType(
        std::string code,
        std::string disease,
        std::string technology,
        int recoveryPeriod)
{
    VaccineType* vt = new VaccineType(code, disease, technology,
         recoveryPeriod);

    bool success = hc->addVaccineType(vt);
    if (!success) {
        delete vt;
    } else {
        FileManager::saveVaccineTypes(hc, "vaccine_types.txt");
    }
    return success;
}

bool VaccineController::registerVaccine(int typeIndex, std::string
     commercialName,
        std::string brand, std::string lot, std::string expiry, int qty) {
    std::vector<VaccineType*> catalog = hc->getVaccineCatalog();

    if (typeIndex < 0 || typeIndex >= catalog.size()) {
        return false;
    }

    VaccineType* selectedType = catalog[typeIndex];
    Vaccine* v = new Vaccine(commercialName, brand, lot, expiry, qty,
         selectedType);
    bool success = hc->addVaccineToInventory(v);

    if (!success) {
        delete v;
        return false;
    }

    FileManager::saveVaccines(hc, "vaccines.txt");
    return true;
}
```

Listing 1.2: Tests for vaccine type and vaccine lot registration

```
TEST(UC1_Tests, RejectDuplicateVaccineType) {
    HealthcareCenter hc("Test Center", "Addr", "123", "email@test.com");
    VaccineController vc(&hc);

    EXPECT_TRUE(vc.createVaccineType("V-001", "COVID-19", "mRNA", 30));
    EXPECT_FALSE(vc.createVaccineType("V-001", "Different Disease",
                                      "Subunit Protein", 45));
    EXPECT_EQ(hc.getVaccineCatalog().size(), 1);
}

TEST(UC2_Tests, RejectInvalidTypeIndex) {
    HealthcareCenter hc("Test Center", "Addr", "123", "email@test.com");
    VaccineController vc(&hc);

    vc.createVaccineType("V-001", "COVID-19", "mRNA", 30);

    EXPECT_FALSE(vc.registerVaccine(1, "Spikevax", "Moderna",
                                    "LOT456", "2026-12-31", 300));
    EXPECT_FALSE(vc.registerVaccine(-1, "Spikevax", "Moderna",
                                    "LOT456", "2026-12-31", 300));
    EXPECT_EQ(hc.getInventory().size(), 0);
}
```

6.2 Appointment Scheduling

The appointment scheduling implementation validates date and time formats, checks whether the SNS user and the Vaccine Type exist, and only persists the appointment if the model accepts it. This prevents appointments for unknown users, unknown vaccine types, or duplicated pending schedules.

Listing 1.3: Appointment scheduling implementation

```
bool AppointmentController::createAppointment(const std::string& snsNumber,
        const std::string& vaccineTypeCode,
        const std::string& date,
        const std::string& time) {
    if (!isValidDate(date) || !isValidTime(time)) {
        return false;
    }

    SNSUser* user = hc->findSNSUserByNumber(snsNumber);
    VaccineType* vaccineType = hc->findVaccineTypeByCode(vaccineTypeCode);
    if (user == nullptr || vaccineType == nullptr) {
        return false;
    }

    Appointment* appointment = new Appointment(user, vaccineType, date, time)
        ;
    bool success = hc->addAppointment(appointment);
    if (!success) {
        delete appointment;
    } else {
        FileManager::saveAppointments(hc, "appointments.txt");
    }

    return success;
}
```

Listing 1.4: Test for successful appointment scheduling

```
TEST(UC7_Tests, ScheduleAppointmentForExistingSNSUserAndVaccineType) {
    HealthcareCenter hc("Test Center", "Addr", "123", "email@test.com");
```

```
    SNSUserController sc(&hc);
    VaccineController vc(&hc);
    AppointmentController ac(&hc);

    ASSERT_TRUE(sc.registerSNSUser("SNS001", "Maria", "Rua A",
                "1990-01-01", "912345678", "12345678"));
    ASSERT_TRUE(vc.createVaccineType("COVID", "COVID-19", "mRNA", 30));

    EXPECT_TRUE(ac.createAppointment("SNS001", "COVID", "2026-06-09", "10:30"
        ));

    ASSERT_EQ(hc.getAppointments().size(), 1);
    EXPECT_EQ(hc.getAppointments()[0]->getUser()->getSnsNumber(), "SNS001");
    EXPECT_EQ(hc.getAppointments()[0]->getVaccineType()->getCode(), "COVID");
    EXPECT_EQ(hc.getAppointments()[0]->getStatus(), "SCHEDULED");
}
```

6.3 Vaccine Administration

The vaccine administration flow is implemented in the model because it changes several domain structures at the same time. The system checks if the selected vaccine exists, if stock is available, if the user has a waiting appointment, and if the vaccine type matches the appointment. If all conditions are valid, the stock is decremented, the user is moved from the waiting room to the recovery room, and the appointment is marked as administered.

Listing 1.5: Vaccine administration implementation

```
bool HealthcareCenter::recordAdministration(const std::string& snsNumber,
        Vaccine* vaccine,
        const std::string& timestamp) {
    if (vaccine == nullptr || vaccine->getQuantity() <= 0) {
        return false;
    }

    Appointment* appointment = findWaitingAppointment(snsNumber);
    if (appointment == nullptr ||
        appointment->getVaccineType()->getCode() != vaccine->getType()->
            getCode()) {
        return false;
    }

    for (auto it = waitingRoom.begin(); it != waitingRoom.end(); ++it) {
        if ((*it)->getSnsNumber() == snsNumber) {
            if (!vaccine->decreaseQuantity()) {
                return false;
            }

            SNSUser* user = *it;
            waitingRoom.erase(it);
            recoveryRoom.push_back(user);
            appointment->markAdministered(vaccine, timestamp);
            return true;
        }
    }

    return false;
}
```

Listing 1.6: Test for vaccine administration side effects

```
TEST(UC10_Tests,
     RecordAdministrationMovesUserToRecoveryAndUpdatesStockAndAppointment) {
    HealthcareCenter hc("Test Center", "Addr", "123", "email@test.com");
    SNSUserController sc(&hc);
    VaccineController vc(&hc);
    AppointmentController ac(&hc);
    NurseController nc(&hc);

    ASSERT_TRUE(sc.registerSNSUser("SNS001", "Maria", "Rua A",
                "1990-01-01", "912345678", "12345678"));
    ASSERT_TRUE(vc.createVaccineType("COVID", "COVID-19", "mRNA", 30));
    ASSERT_TRUE(vc.registerVaccine(0, "Comirnaty", "Pfizer",
                "LOT123", "2027-12-31", 5));
    ASSERT_TRUE(ac.createAppointment("SNS001", "COVID",
                AppointmentController::currentDate(), "10:30"));
    ASSERT_TRUE(ac.registerArrival("SNS001"));

    EXPECT_TRUE(nc.recordAdministration("SNS001", "LOT123"));

    EXPECT_EQ(hc.getWaitingRoom().size(), 0);
    ASSERT_EQ(hc.getRecoveryRoom().size(), 1);
    EXPECT_EQ(hc.getRecoveryRoom()[0]->getSnsNumber(), "SNS001");
    EXPECT_EQ(hc.getInventory()[0]->getQuantity(), 4);
    EXPECT_EQ(hc.getAppointments()[0]->getStatus(), "ADMINISTERED");
}
```

6.4 Data Validation

The system includes reusable validation functions for phone numbers, citizen card numbers, SNS numbers, dates, times, and emails. These functions support the controllers and views by keeping input rules centralized.

Listing 1.7: Date and time validation utilities

```
bool isValidDate(const std::string& date) {
    if (date.length() != 10 || date[4] != '-' || date[7] != '-') return false
        ;
    for (int i = 0; i < 10; ++i) {
        if (i == 4 || i == 7) continue;
        if (!std::isdigit(date[i])) return false;
    }
    return true;
}

bool isValidTime(const std::string& time) {
    if (time.length() != 5 || time[2] != ':') return false;
    for (int i = 0; i < 5; ++i) {
        if (i == 2) continue;
        if (!std::isdigit(time[i])) return false;
    }

    int hour = std::stoi(time.substr(0, 2));
    int minute = std::stoi(time.substr(3, 2));
    return hour >= 0 && hour <= 23 && minute >= 0 && minute <= 59;
}
```

Listing 1.8: Tests for validation utilities

```
TEST(Utils_Tests, ValidateDate) {
    EXPECT_TRUE(isValidDate("2026-12-31"));
    EXPECT_FALSE(isValidDate("amanha"));
    EXPECT_FALSE(isValidDate("2026/12/31"));
}
```

```
6
7  TEST(Utils_Tests, ValidatePhone) {
8      EXPECT_TRUE(isValidPhone("912345678"));
9      EXPECT_FALSE(isValidPhone("123456789"));
10     EXPECT_FALSE(isValidPhone("91234567a"));
11 }
```

6.5 Validation Improvement: Vaccine Expiry Date

The current implementation validates generic date formatting, but the vaccine registration flow should be improved to validate the expiry date specifically. The expected rule is that a vaccine lot can only be registered when the expiry date has a valid format and is not earlier than the current date. This improvement should be added to UC2, to the corresponding Sequence Diagram, and to the GoogleTest suite.

Listing 1.9: Suggested additional test for expired vaccine lots

```
1  TEST(UC2_Tests, RejectExpiredVaccineLot) {
2      HealthcareCenter hc("Test Center", "Addr", "123", "email@test.com");
3      VaccineController vc(&hc);
4
5      ASSERT_TRUE(vc.createVaccineType("V-001", "COVID-19", "mRNA", 30));
6
7      EXPECT_FALSE(vc.registerVaccine(0, "Comirnaty", "Pfizer",
8                                      "OLDLOT", "2020-01-01", 500));
9      EXPECT_EQ(hc.getInventory().size(), 0);
10 }
```

7 Conclusion

This report presented the final state of the MedManager project at the conclusion of its development cycle. The system successfully implements the core operational workflows of a vaccination center, including vaccine type management, vaccine inventory registration, employee and SNS user management, appointment scheduling, arrival registration, waiting-room control, vaccine administration, recovery-room monitoring, data persistence, and report generation.

The project follows a structured architecture based on the separation of Model, View, and Controller responsibilities, promoting maintainability and facilitating future extensions. The Domain Model, Glossary, Class Diagram, Use Case Diagrams, and Sequence Diagrams provide a clear and accurate representation of both the business domain and the implemented solution.

The inclusion of automated tests using GoogleTest has significantly contributed to ensuring software quality by validating expected execution flows and error-handling scenarios. The presented test suite demonstrates the reliability of critical functionalities such as registration processes, appointment scheduling, vaccine administration, and data validation.

Although this release delivers a robust and mature implementation, a minor limitation remains regarding the active blocking of expired vaccine lots during registration. While the date format is correctly validated by the system, strict expiry enforcement is left as a potential future enhancement.

Overall, the MedManager project establishes a solid, complete, and functional foundation for vaccination center management. It successfully demonstrates the practical application of software engineering principles throughout the analysis, design, implementation, and testing phases of development.

6 References

1. P. B. Sousa, "Lecture Slides for FSOFT (Fundamentals of Software Engineering)," Lecture notes and presentation slides, Bachelor in Telecommunications and Informatics Engineering, ISEP (School of Engineering, Polytechnic of Porto), 2025–2026. Provided by the instructor.